结构型模式

适配器模式（Adapter）

**定义：将一个接口转换成客户希望的另一个接口，适配器模式使接口不兼容的那些类可以一起工作。

动机：当客户类调用适配器的方法时，在适配器类的内部将调用适配者类的方法，而这个过程对客户类是透明的，客户类并不直接访问适配者类。因此，适配器可以使由于接口不兼容而不能交互的类可以一起工作，这就是适配器模式的模式动机。

模式结构：Target：目标抽象类 Adapter：适配器类 Adaptee：适配者类

代码：

```
// 典型的类适配器

public class Adapter extends Adaptee implements Target {
    public void request() {
        super.specificRequest();
    }
}
// 继承现有的类调用它的方法去实现目标类的方法

// 典型的对象适配器
public class Adapter extends/* 或者implements */ Target {
    private Adaptee adaptee;      //维持一个对适配者对象的引用

    public Adapter(Adaptee adaptee) {
        this.adaptee=adaptee;
    }
    public void request() {
        adaptee.specificRequest(); //转发调用
```

```
19          }
20      }
```

优点：

1. 将目标类和适配者类解耦，通过引入一个适配器类来重用现有的适配者类，无须修改原有结构
2. 增加了类的透明性和复用性，提高了适配者的复用性，同一个适配者类可以在多个不同的系统中复用
3. 灵活性和扩展性非常好，更换(增加)适配器，符合开闭原则。

类适配器模式：由于继承关系，置换一些适配者的方法很方便

对象适配器模式：可以把多个不同的适配者适配到同一个目标，还可以适配一个适配者的子类

缺点：

类适配器模式：

1. 一次最多只能适配一个适配者类，不能同时适配多个适配者
2. 适配者类不能为最终类（final）
3. 目标抽象类只能为接口，不能为类

对象适配器模式：

1. 在适配器中置换适配者类的某些方法比较麻烦

适用环境：

1. 系统需要使用一些现有的类，而这些类的接口不符合系统的需要，甚至没有这些类的源代码

> 2. 创建一个可以重复使用的类，用于和一些彼此之间没有太大关联的类，包括一些可能在将来引进的类一起工作

缺省适配器：当不需要实现一个接口所提供的所有方法时，可先设计一个抽象类实现该接口，并为接口中每个方法提供一个默认实现（空方法），那么该抽象类的子类可以选择性地覆盖父类的某些方法来实现需求，它适用于不想使用一个接口中的所有方法的情况，又称为单接口适配器模式。

```
public abstract class AbstractServiceClass implements
ServiceInterface {
    public void serviceMethod1() {  }  //空方法
    public void serviceMethod2() {  }  //空方法
    public void serviceMethod3() {  }  //空方法
}
```

双向适配器：

```
public class Adapter implements Target,Adaptee {
    private Target target;
    private Adaptee adaptee;

    public Adapter(Target target) {
        this.target = target;
    }
    public Adapter(Adaptee adaptee) {
        this.adaptee = adaptee;
    }
    public void request() {
        // 互相调用对方的方法
    }
    public void specificRequest() {
        // 互相调用对方的方法
    }
}

```

桥接模式（Bridge）

定义：将抽象部分与它的实现部分分离，使它们都可以独立地变化。用抽象关联取代了传统的多层继承，将类之间的静态继承关系转换为动态的对象组合关系。

动机：

模式结构：Abstraction：抽象类 RefinedAbstraction：细化抽象类 Implementor：实现类接口 ConcreteImplementor：具体实现类

代码：

```
// 实现类接口代码
public interface Implementor {
     public void operationImpl();
}
// 具体实现类代码
public class ConcreteImplementor implements Implementor {
      public void operationImpl(){
        //具体业务方法的实现
      }
}
// 典型的抽象类代码
public abstract class Abstraction{
    protected Implementor impl;

    public void AsetImpl(Implementor impl) {
        this.impl=impl;
    }

    public abstract void operation();
}
// 典型的细化抽象类
public class RefinedAbstraction extends Abstraction {

       public abstract void operation()
```

```
25          {
26            //    代码
27             impl.operation();
28             //代码
29          }
30  }
31
```

优点：

1. 分离抽象接口及其实现部分
2. 可以取代多层继承方案，极大地减少了子类的个数
3. 提高了系统的可扩展性，在两个变化维度中任意扩展一个维度，不需要修改原有系统，符合开闭原则

缺点：

1. 会增加系统的理解与设计难度，由于关联关系建立在抽象层，要求开发者一开始就要针对抽象层进行设计与编程
2. 正确识别出系统中两个独立变化的维度并不是一件容易的事情

适用环境：

1. 需要在抽象化和具体化之间增加更多的灵活性，避免在两个层次之间建立静态的继承关系
2. 抽象部分和实现部分可以以继承的方式独立扩展而互不影响
3. 一个类存在两个（或多个）独立变化的维度，且这两个（或多个）维度都需要独立地进行扩展
4. 不希望使用继承或因为多层继承导致系统类的个数急剧增加的系统

组合模式（Composite）

定义：组合多个对象形成树形结构以表示“部分-整体”的结构层次。组合模式对单个对象（即叶子对象）和组合对象（即容器对象）的使用具有一致性。

动机：如何将容器对象和叶子对象进行递归组合，使得用户在使用时无须对它们进行区分，可以一致地对待容器对象和叶子对象

模式结构：Component：抽象构件类 Leaf：叶子构件 Composite：容器构件

代码：

```
// 抽象构件示例代码
public abstract class Component {
    public abstract void add(Component c); //增加成员
    public abstract void remove(Component c); //删除成员
    public abstract Component getChild(int i); //获取成员
    public abstract void operation();  //业务方法
}

// 叶子构件示例代码
public class Leaf extends Component {
    public void add(Component c) {
        //异常处理或错误提示    }
    public void remove(Component c) {
        //异常处理或错误提示    }
    public Component getChild(int i) {
        //异常处理或错误提示
        return null;     }
    public void operation() {
        //叶子构件具体业务方法的实现    }
}

// 容器构件
public class Composite extends Component {
```

```
24      private ArrayList<Component> list = new ArrayList<Component>(
    );
25      public void add(Component c) {
26          list.add(c);
27      }
28      public void remove(Component c) {
29          list.remove(c);    }
30      public Component getChild(int i) {
31          return (Component)list.get(i);    }
32      public void operation() {
33          //容器构件具体业务方法的实现，将递归调用成员构件的业务方法
34          for(Object obj:list) {
35              ((Component)obj).operation();
36          }
37      }
38  }
```

优点：

1. 可以清楚地定义分层次的复杂对象，表示对象的全部或部分层次，让客户端忽略了层次的差异，方便对整个层次结构进行控制
2. 客户端可以一致地使用一个组合结构或其中单个对象，不必关心处理的是单个对象还是整个组合结构，简化了客户端代码
3. 增加新的容器构件和叶子构件都很方便，符合开闭原则
4. 为树形结构的面向对象实现提供了一种灵活的解决方案

缺点：

1. 设计更加抽象，对象的业务规则如果很复杂，实现组合模式具有较大难度。而且不是所有的方法都与叶子对象子类都有关联。
2. 在增加新构件时很难对容器中的构件类型进行限制

适用环境：

> 1. 在具有整体和部分的层次结构中，希望通过一种方式忽略整体与部分的差异，客户端可以一致地对待它们
> 2. 在一个使用面向对象语言开发的系统中需要处理一个树形结构
> 3. 在一个系统中能够分离出叶子对象和容器对象，而且它们的类型不固定，需要增加一些新的类型

透明组合模式：抽象构件Component中声明了所有用于管理成员对象的方法，包括add()、remove()，以及getChild()等方法

> 优点：确保所有的构件类都有相同的接口
>
> 缺点：是不够安全，因为叶子对象和容器对象在本质上是有区别的

安全组合模式：抽象构件Component中没有声明任何用于管理成员对象的方法，而是在Composite类中声明并实现这些方法

> 优点：安全对于叶子对象，客户端不可能调用到这些方法
>
> 缺点：是不够透明，客户端不能完全针对抽象编程，必须有区别地对待叶子构件和容器构件使用频率更高

装饰模式（Decorator）

适用环境：

定义：在不改变一个对象本身功能的基础上给对象增加额外的新行为

动机：在不改变一个对象本身功能的基础上给对象增加额外的新行为

模式结构：Component：抽象构件类 ConcreteComponent：具体构件类 Decorator：抽象装饰类 ConcreteDecorator：具体装饰类

代码：

```java
// 抽象构件
public abstract class Component {
    public abstract void operation();
}
// 具体构件
public class ConcreteComponent  extends Component {
    public void operation() {
        //实现基本功能
    }
}

// 抽象装饰类
public class Decorator extends Component {
    private Component component; //维持一个对抽象构件对象的引用

    //注入一个抽象构件类型的对象
    public Decorator(Component component) {
        this.component=component;
    }
    public void operation() {
        component.operation();  //调用原有业务方法
    }
}

// 具体装饰类
public class ConcreteDecorator extends Decorator {
    public ConcreteDecorator(Component component) {
        super(component);
    }
    public void operation() {
        super.operation(); //调用原有业务方法
        addedBehavior(); //调用新增业务方法
    }
    //新增业务方法
    public void addedBehavior() {
        ……
    }
```

```
38  }
39
```

优点：

1. 对于扩展一个对象的功能，装饰模式比继承更加灵活，不会导致类的个数急剧增加
2. 可以通过一种动态的方式来扩展一个对象的功能，通过配置文件可以在运行时选择不同的具体装饰类，从而实现不同的行为
3. 可以对一个对象进行多次装饰
4. 具体构件类与具体装饰类可以独立变化，用户可以根据需要增加新的具体构件类和具体装饰类，且原有类库代码无须改变，符合开闭原则

缺点：

1. 使用装饰模式进行系统设计时将产生很多小对象，大量小对象的产生势必会占用更多的系统资源，在一定程度上影响程序的性能
2. 比继承更加易于出错，排错也更困难，对于多次装饰的对象，调试时寻找错误可能需要逐级排查，较为烦琐

适用环境：

1. 在不影响其他对象的情况下，以动态、透明的方式给单个对象添加职责
2. 当不能采用继承的方式对系统进行扩展或者采用继承不利于系统扩展和维护时可以使用装饰模式

透明装饰模式：客户端完全针对抽象编程，装饰模式的透明性要求客户端程序不应该将对象声明为具体构件类型或具体装饰类型，而应该全部声明为抽象构件类型。可以对一个已装饰过的对象进行多次装饰，得到更为复杂、功能更为强大的对象无法在客户端单独调用新增方法addedBehavior()

```
Component component_o,component_d1,component_d2;
                            //全部使用抽象构件定义
component_o = new ConcreteComponent();
component_d1 = new ConcreteDecorator1(component_o);
component_d2 = new ConcreteDecorator2(component_d1);
component_d2.operation();
              //无法单独调用component_d2的addedBehavior()方法

```

半透明装饰模式：用具体装饰类型来定义装饰之后的对象，而具体构件使用抽象构件类型来定义，客户端使用具体装饰类型来定义装饰后的对象，因此可以单独调用addedBehavior()方法。

```
Component component_o; //使用抽象构件类型定义
component_o = new ConcreteComponent();
component_o.operation();
ConcreteDecorator component_d; //使用具体装饰类型定义
component_d = new ConcreteDecorator(component_o);
component_d.operation();
component_d.addedBehavior(); //单独调用新增业务方法
```

外观模式（Facade）

定义：外部与子系统的通信通过一个统一的外观对象进行，为子系统中的一组接口提供一个统一的入口

动机：一个客户类需要和多个业务类交互，有时候这些需要交互的业务类会作为一个整体出现

模式结构：Facade：外观类 SubSystem：子系统角色

代码：

```
// 外观子系统
```

```
public class SubSystemA {
    public void methodA() {
        //业务实现代码
    }
}
public class SubSystemB {
    public void methodB() {
        //业务实现代码
    }
}

// 外观类
public class Facade {

    private SubSystemA obj1 = new SubSystemA();
    private SubSystemB obj2 = new SubSystemB();
    private SubSystemC obj3 = new SubSystemC();

    public void method() {
        obj1.method();
        obj2.method();
        obj3.method();
    }
}

// 客户端
public class Client {
    public static void main(String args[]) {
        Facade facade = new Facade();
        facade.method();
    }
}
```

优点：

> 1. 它对客户端屏蔽了子系统组件，减少了客户端所需处理的对象数目，并使得子系统使用起来更加容易

2. 它实现了子系统与客户端之间的松耦合关系，这使得子系统的变化不会影响到调用它的客户端，只需要调整外观类即可
3. 一个子系统的修改对其他子系统没有任何影响，而且子系统的内部变化也不会影响到外观对象

缺点：

1. 不能很好地限制客户端直接使用子系统类，如果对客户端访问子系统类做太多的限制则减少了可变性和灵活性
2. 如果设计不当，增加新的子系统可能需要修改外观类的源代码，违背了开闭原则

适用环境：

1. 要为访问一系列复杂的子系统提供一个简单入口
2. 客户端程序与多个子系统之间存在很大的依赖性
3. 在层次化结构中，可以使用外观模式定义系统中每一层的入口，层与层之间不直接产生联系，而是通过外观类建立联系，降低层之间的耦合度

代理模式（Proxy）

定义：给某一个对象提供一个代理，并由代理对象控制对原对象的引用

动机：通过引入一个新的对象（如小图片和远程代理对象）来实现对真实对象的操作，或者将新的对象作为真实对象的一个替身

模式结构：Subject：抽象主题角色 Proxy：代理主题角色 RealSubject：真实主题角色

代码：

```
// 抽象主题类示例代码
public abstract class Subject {
    public abstract void request();
}
```

```
// 真实主题类示例代码
public class RealSubject extends Subject{
    public void request() {
          //业务方法具体实现代码
    }
}
// 代理类示例代码
public class Proxy extends Subject {
    private RealSubject realSubject = new RealSubject();
                                //维持一个对真实主题对象的引用
    public void preRequest() {  …...    }
    public void request() {
        preRequest();
        realSubject.request();  //调用真实主题对象的方法
        postRequest();
      }
    public void postRequest() {     ……    }
}

```

优点：

1. 能够协调调用者和被调用者，在一定程度上降低了系统的耦合度
2. 客户端可以针对抽象主题角色进行编程，增加和更换代理类无须修改源代码，符合开闭原则，系统具有较好的灵活性和可扩展性

缺点：

1. 由于在客户端和真实主题之间增加了代理对象，因此有些类型的代理模式可能会造成请求的处理速度变慢（例如保护代理）
2. 实现代理模式需要额外的工作，而且有些代理模式的实现过程较为复杂（例如远程代理）

适用环境：

1. 当客户端对象需要访问远程主机中的对象时可以使用远程代理
2. 当需要用一个消耗资源较少的对象来代表一个消耗资源较多的对象，从而降低系统开销、缩短运行时间时可以使用虚拟代理
3. 当需要为某一个被频繁访问的操作结果提供一个临时存储空间，以供多个客户端共享访问这些结果时可以使用缓冲代理
4. 当需要控制对一个对象的访问，为不同用户提供不同级别的访问权限时可以使用保护代理
5. 当需要为一个对象的访问（引用）提供一些额外的操作时可以使用智能引用代理